Intelligent Emergency Evacuation Route Planning Using A* Search

Kadali Nikshita
BL.EN.U4CSE23128

October 2026

1 Problem Domain and Statement

In case of an emergency such as fire, natural disaster, or any other disaster, the occupants must urgently and effectively reach an emergency exit from the building. However, the task of finding the safest way may not be easy since in such an event there might be blocked and overcrowded corridors or other hazards.
The proposed system simulates the building by a graph; the rooms, corridors, and places of breaks are considered respectively as vertices and edges of the graph. The agent is defined as an intelligent agent that starts from its current position with the objective to reach an exit safely. The A* algorithm is used to find an optimal path considering both already traversed distance and remaining distance to the exit. Once some paths are blocked or unsafe, a route without the obstacles can be searched. The system takes into account the different kinds of individuals inside the construction as separate intelligent agents. There are particular roles assigned to the agents depending on the type of individuals being evacuated.

1.1 Agents

The agents are as below:

- Agent 1 → Student
- Agent 2 → Student
- Agent 3 → Staff
- Agent 4 → Visitor

1.2 Agents Goals

The key details that are recorded by the agent include:

- Current Position – The agent's current place in the building.
- Goal – The emergency exit which the agent must reach safely.
- Path – The path created on the basis of the A* algorithm.
- Movement – The travel of the agent in the current place.
- Status – The agent's status at that moment: Evacuating, Reached Exit, or Blocked.

1.3 How do agents interact with each other

Interaction among agents includes the sharing of information regarding their locations and planned movements. Each agent has to determine whether another agent is already or is about to occupy the same location. In this case, detection of any possible collisions or congestion makes the agents take actions such as waiting, changing the planned route, or opening another exit.

2 PEAS

- **Performance Measure:** Minimum path cost, minimum evacuation time, avoid danger, successfully reach exit.
- **Environment:** Building/grid with corridors, obstacles, fire zones and exits.
- **Actuators:** Agent movement: up, down, left, right.
- **Sensors:** Agent's position, nearby obstacles, danger zones, available exits.

3 Environment Properties

4 Type of Agent

Goal-Based Agent with Utility-Based Decision Making
Every evacuee is assigned a Goal-Based Agent, which employs the A* search algorithm towards the goal of taking the best route possible towards a safe exit. The agent evaluates its current position, existing obstacles, danger zones, other agents, and available exits to figure out its subsequent action. A goal-based agent is appropriate because evacuation is about the continuous choice of actions that allow the agent to move closer to the goal while steering clear of dangers.

Table 1: Environment Properties

Property	**Our System**	**Why**
Observable	Partially observable	Agent may only know relevant/current environment information.
Deterministic/Stochastic	Stochastic	Danger and blockages can change.
Episodic/Sequential	Sequential	Every movement affects the next decision.
Static/Dynamic	Dynamic	Fire, danger, or blocked paths can change.
Discrete/Continuous	Discrete	Grid cells and movements are discrete.
Single/Multi-agent	Multi-agent	Multiple people evacuate simultaneously.

4.1 Cooperative or Competitive

The agents are working together as a team rather than competing against each other. All of the agents work toward the same goal of implementing a safe evacuation from the premise. They exchange information regarding their position, thus avoiding any collisions or traffic jams. If two agents want to pass through the same channel, one of the agents can either wait for the other to pass or take another route.

5 Searching Technique

5.1 A*

We utilize A* due to its ability to blend combined costs of straight distance already travelled with estimated distance to reach the target zone, which assists the agent to identify the best route for escape. Because the agents are using a grid to move up or down or horizontally, Manhattan Distance is an appropriate heuristic that can predict the distance left to reach the target. A* uses the evaluation function:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the actual cost from the starting position to node n, and $h(n)$ is the estimated cost from node n to the goal.

For our grid-based environment, we use Manhattan Distance as the heuristic:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

The returned path is

$$(6, 16) \to (7, 16) \to (7, 17) \to (7, 18) \to (7, 19),$$

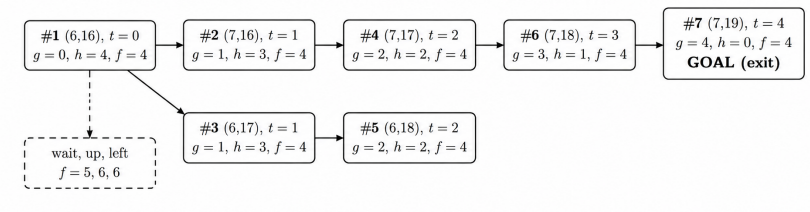


Figure 1: Search Tree and Sequence of States Explored Using A*

which is a 4-step route. Nodes #3 and #5 lead to an equally good route via row 6, but A* reaches the goal first through nodes #4 and #6 because of the tie-breaking order.

6 Working Cases and Edge Cases

All outputs presented below are generated by `evacuation.py` using the `A*` algorithm. The snapshot legend is as follows: `#` represents a wall, a digit represents the agent ID, `*` represents fire, `E` represents an emergency exit, and `.` represents a floor cell. The fire spreads by one cell every two time steps, while in the bottleneck scenario, it spreads by one cell every three time steps.

6.1 Working Case 1: Normal Evacuation (4 Agents)

Fire starts at $(5, 4)$, in the lower-left room. All four agents are saved, with a total time of $t = 21$ and a total of 341 expanded nodes.

$t = 0$

```
####################
#459.......#.......#
#670.......#...2...#
#8..................#
#####.######.#######
#...*.......#.......#
#...1.......#....3..#
E..........#.......E
####################
```

$t = 18$

```
####################
#...***....#.......#
#..*****...#.......#
#.*******..........#
######*######.#######
#**********9.|.....#
#*********.#7......#
*********..#64580..E
####################
```

Figure 2: Normal evacuation scenario at $t = 0$ and $t = 14$.

6.1.1 What Happens and Why

- Agent 1 starts next to the fire but is 5 steps from the left exit and gets out immediately (5 steps).

- Agent 0 starts in the upper-left room. The lower door is right beside the fire, and the penalty makes that route expensive, so it travels through the middle door to the right exit instead (21 steps). A greedy "nearest exit" rule would send it left, where the fire is.
- Agents 2 and 3 are in the right half, away from the fire, and use the right exit (15 and 4 steps). They do not interfere with Agent 0 because their reservations are time-separated.

6.2 Working Case 2: Bottleneck with 10 Agents

Six extra agents are added in the upper-left room, and the fire spreads every 3 steps. All 10 agents are saved at $t = 27$, with 3,663 total expanded nodes. This case needs the retry mechanism: with the original priority order, Agent 8 (start $(3, 1)$) was boxed in by higher-priority agents and was reported as having no route. After promoting it to the front of the order, a valid plan exists.

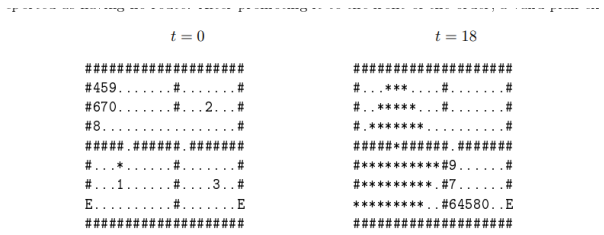


Figure 3: Bottleneck evacuation scenario at $t = 0$ and $t = 18$.

6.2.1 What Happens and Why

The upper-left room has a single doorway towards the right half, so the agents form a line and pass one after another. The waiting action lets them queue instead of colliding. By $t = 18$, the queue of agents $6, 4, 5, 8, 0$ is moving along the bottom row of the right room towards the exit at $(7, 19)$, while the fire grows in the left half. Exit capacity is one agent per time step, which is why later agents take 22–27 steps.

6.3 Edge Case 1: Nearest Exit is Blocked by Fire

Fire starts at $(7, 2)$, two cells from the left exit, which ignites at $t = 4$. Agent 1 (start $(6, 4)$) would normally exit left in 5 steps, but it could only arrive at $t = 5$, when the exit is already burning. All four agents are still saved, with a total time of $t = 22$.

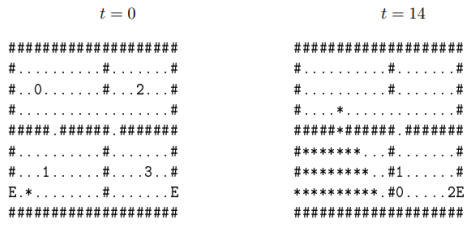


Figure 4: Edge Case 1: Nearest exit blocked by fire.

6.3.1 What Happens and Why

The search treats a burning exit as a wall at that time, so no plan through the left exit exists. A* automatically falls back to the right exit: Agent 1 now needs 22 steps instead of 5, and Agent 0 follows the same alternative route. This shows that the planner adapts to a changed goal set without requiring special-case code.

6.4 Edge Case 2: Agent Trapped (No Safe Route)

Fire starts at both doors of the upper-left room, $(4, 5)$ and $(3, 11)$, and does not spread further. Agent 0 (start $(2, 3)$) is sealed in. The planner reports `NO SAFE ROUTE` after exhausting all reachable $(cell, time)$ states (2,317 expansions), and the remaining 3 agents are evacuated normally (3 of 4 saved).

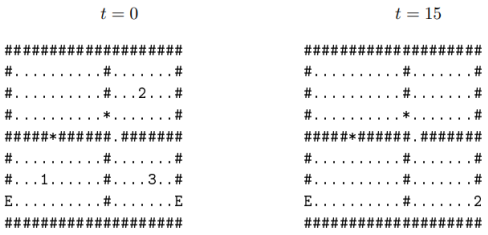


Figure 5: Edge Case 2: Agent trapped with no safe route at $t = 0$ and $t = 15$.

6.4.1 What Happens and Why

The goal is unreachable, so the open list eventually empties (the search is bounded by the horizon $T_{\max} = 80$) and the algorithm terminates correctly instead of looping. The trapped agent is not drawn because no plan exists; it stays in the upper-left room. In a real system, this is the signal to alert rescuers. This case also shows the cost of failure: proving "no path" is much more expensive (2,317 nodes) than finding one (7 to 300 nodes).

7 Complexity Understanding

Let b be the branching factor, d the depth of the solution (number of time steps), V the number of free cells, T the time horizon, and n the number of agents.

- **Branching factor:** $b = 5$ (4 moves + wait), reduced in practice by walls, fire, and reservations.
- **General worst case of A*:** time $O(b^d)$ and space $O(b^d)$. For a solution depth of $d = 27$, this is $5^{27} \approx 7 \times 10^{18}$ nodes, which is impossible to search blindly.
- **Bound for our problem.** Because states are (cell, time) pairs and repeated states are pruned by the best-g table, one agent has at most $V \cdot (T+1)$ distinct states. Here $V = 107$ free cells and $T = 80$, so at most ≈ 8667 states. With a binary heap, time is $O(VT \log(VT))$ per agent and space is $O(VT)$.
- **For n agents (sequential planning):** $O(nVT \log(VT))$ in total, multiplied by up to 4 if the retry loop runs.
- **Role of the heuristic.** With an accurate h, A* expands only nodes with $f \leq C^*$, which is far fewer than the bound. Agent 3 needs 7 expansions for a depth-4 solution; the largest successful search (Agent 0 in the normal case, depth 21 after queueing and fire avoidance) needs 204.

Relation to the problem. The greater the building size and the waiting time in queues, the more the depth d grows. It is assumed that $b = 5$. As the building grows, the value of V also increases. When the number of agents increases, they can prevent each other from moving, causing the depth to increase due to additional waiting time. This makes bottleneck cases more expensive: agents 4 and 9 in bottleneck runs require approximately 230–310 expansions each, whereas agents 7 and 9 in cases without a bottleneck require fewer than 10 expansions.

8 Comparison and Conceptual Understanding

We compared A* with two alternatives by changing only the priority function in the same planner: UCS ($f = g$, no heuristic, equivalent to Dijkstra / uniform-cost BFS) and Greedy Best-First ($f = h$).

8.1 Search Process

- **UCS:** UCS expands states in order of cost g only. It spreads out in all directions through the $(cell, time)$ space, including a way from the exits, until it reaches the goal. It is optimal and complete but ignores where the exits are.

Table 2: Comparison of Search Algorithms Across Evacuation Scenarios

Scenario	Algorithm	Nodes	Time	Saved
Normal	A* / Greedy	341 / 693	21	4/4
Exit blocked	A* / Greedy	779 / 1763	22	4/4
Bottleneck	A* / Greedy	3663 / 6017	27	10/10
Trapped	A* / Greedy	9816 / 14116	15	3/4
Fast fire	A* / Greedy	896 / 1236	15	3/4
Door fire	A* / Greedy	1068 / 800	12	3/4
Crowd + fire	A* / Greedy	3550 / 3081	14	5/10

- **Greedy:** Greedy always expands the state that looks closest to an exit using h only. It moves quickly towards an exit, but it does not account for the cost already paid.
- **A*:** A* uses $g + h$: it combines the cost so far with an estimate of the remaining cost, so it is guided towards the exits and is still optimal.

8.2 Solution Quality

UCS and A* are optimal given the same reservations. Greedy is not guaranteed to be optimal: with fire penalties and walls it can commit to a route that looks short but is costly (for example, a dead end or the next exit to the fire). It happened to match A* in these runs because the maps are simple, but this is not guaranteed.

8.3 Limitations

- **UCS:** Explores too many states; the cost grows quickly with map size.
- **Greedy:** No optimality guarantee; it can be misled by a bad heuristic, such as dead ends or walls.
- **A*:** Memory grows with $V \cdot T$; the quality depends on the heuristic. The Manhattan heuristic ignores walls and fire, so it is weak in a maze-like building.
- **Prioritized planning (all three):** It is not complete and not globally optimal; a low-priority agent can be boxed in by agents ahead of it. The order affects total evacuation time. We reduced this with the retry step, but a guaranteed solution would require a more complete method such as Conflict-Based Search (CBS).

8.4 What Happens as the Search Space Grows

UCS grows roughly with the number of cells because the cost radius is $O(d)$ in the worst case; Greedy and A* are directed, so they grow much more slowly. In

the table, UCS needs 3.3× and 2.8× as many expansions as A*, and the gap widens as more agents are added because the area and reservation space grow faster than the length of the route.

Adding agents also enlarges the search per agent because more reserved cells, more waiting, and a larger d increase the horizon T. This becomes the main cost driver. Practical ways to scale are a better heuristic (true shortest distance ignoring agents, computed once by reverse BFS), a smaller planning horizon, hierarchical planning by rooms, and replanning only when the fire changes.

9 Conclusion

We developed a multi-agent evacuation planner based on a utility-aware, goal-oriented agent using space-time A* with cooperation reservations. The treatment of time as part of the state enables agents to operate in a dynamic environment by waiting in queues, dodging the cells getting burnt, and making a detour when the exit is congested. The actual comparison shows that A* has the same optimal evacuation times as UCS but expands around 2.8-3.3 times fewer nodes. It is also seen that A* handles gracefully edge cases when no one is able to reach the exit. The major drawback is that priority-based planning is not complete. The retry phase addresses the situations which we described above, and using Conflict-Based Search would be a good follow-up for complete assurance.